

World Models for Control across the Representation Spectrum: Transfer and Planning with R2-Dreamer and DINO-WM under a Commodity-GPU Budget

Thomas Georges*

June 16, 2026 — v1.0 (combined report)

Abstract

We study how to *obtain* and how to *use* a world model for control under a strict commodity-GPU budget, at two opposite choices of latent representation. In the first track, a reconstruction-free, representation-regularized DreamerV3 variant (“R2-Dreamer” [18, 1]) *learns* its latent end-to-end, and we measure transfer on a visual locomotion shift on the DeepMind Control Suite [15] (`walker_walk` → `walker_run` from 64×64 pixels). In the second, DINO-WM [8] *inherits* a frozen self-supervised representation (DINOv2 patch features [9]) and learns only an action-conditioned dynamics predictor; we reproduce it on PointMaze, add a low-data adaptation study, and build a coupled local/global planning study on the resulting model. The two tracks bracket the representation axis — learn-your-own versus inherit-frozen — and the engineering thread that makes both feasible on free Colab hardware is shared (a $\approx 21\times$ latent-cache speedup is what makes the DINO-WM study runnable at all). Across both ends *two findings recur*. (i) **Fine-tuning beats training from scratch at equal target-task budget, at both ends** — though *what* the warm start tests differs: Track I is a genuine task shift (`walker_walk`→`walker_run`), where R2-Dreamer reaches 742 vs 563 episodic return (+32%, with a $\sim 4\times$ mid-training lead and a near-zero-shot ~ 375 vs ~ 25 warm start); Track II is same-domain low-data adaptation (a full-data PointMaze checkpoint adapted on 300 target rollouts), where the adapted predictor delivers $2.75\times$ the scratch baseline’s real-environment planning success (0.275 vs 0.10; episode-level $z=4.5$, $p < 10^{-5}$, not a run-level test — all runs are seed 0), reaches a planning score similar to the full-data checkpoint under this evaluation, and converges in a single epoch. (ii) **The scalar metric the model is trained on systematically understates a structural difference in the latent that actually governs control**: two same-task R2-Dreamer policies occupy distinct, near-linearly-separable latent basins (the fine-tune traces a limit-cycle manifold, scratch collapses to a blob); two DINO-WM predictors with the *same order of magnitude* of latent-prediction MSE differ $2.75\times$ in planning success; and a local surrogate whose one-step disagreement with the global model is only ≈ 0.002 nonetheless yields gradient directions reliable enough to be accepted just 2.8% of the time. On the planner side, a global re-score gate keeps cheap first-order refinement *bounded-worsening* under the global model’s objective (rejecting any refinement that worsens it by more than $\tau=0.05$), and here recovers global-CEM-quality plans (0.97 success) where an ungarded hybrid collapses to 0.54, while purely local planners run 100–1000 $\times$ cheaper per episode. All results are single-seed and at fixed budgets; we state the resulting limits throughout.

1 Introduction

A world model is useful for control to the extent that planning or acting against it produces behavior that succeeds in the real environment. Two design questions dominate the cost of

*Engineering and experiment automation developed with Claude (Anthropic) in the `wm-prediction` repository. Track I code: `scripts/r2dreamer`, `configs/r2dreamer`; Tracks II–III code: `src/wm_poc/dino_wm`, `src/wm_poc/local_global`. Every number and figure is read from the committed outputs of the results notebooks (06/07/08).

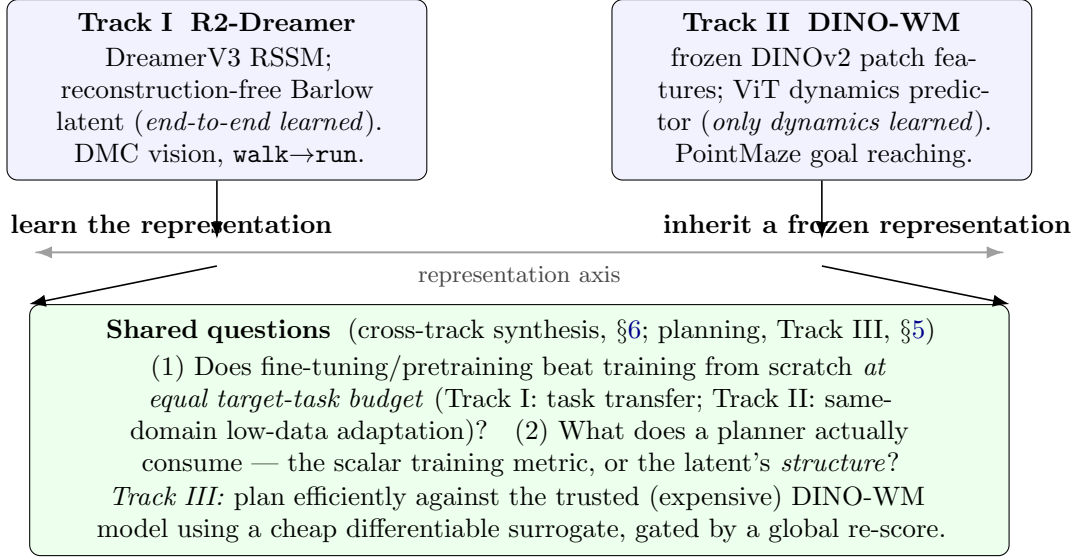


Figure 1: The logic of the repository. The two experimental tracks bracket the representation axis: R2-Dreamer learns its latent end-to-end (Track I, §3); DINO-WM inherits a frozen DINOv2 latent and learns only dynamics (Track II, §4). Both are then asked the same two questions, and a third track (§5) studies efficient planning against the DINO-WM model. The cross-track synthesis (§6) shows that the same two answers hold at both ends.

getting there: *what representation does the model’s dynamics operate over*, and *how is that model adapted to a new task*. This report collects the two experimental tracks of the **wm-prediction** repository, which were built to probe exactly those two questions from opposite ends, and unifies them into a single study.

1.1 The representation spectrum, and the logic of the two tracks

World models for control sit on a spectrum defined by *where the representation comes from* (Figure 1). At one end, the model **learns its own latent** jointly with the dynamics — the DreamerV3 lineage [3, 2, 1], here in its reconstruction-free R2-Dreamer form, which replaces the pixel decoder with a redundancy-reduction representation objective [6]. At the other end, the model **inherits a frozen, pretrained representation** and learns only how it evolves under actions — DINO-WM [8], which trains an action-conditioned transition model over frozen DINOv2 patch features [9] and never updates the encoder. The first buys a control-shaped latent at the cost of learning perception from scratch; the second buys data- and compute-efficiency and cross-task reuse at the cost of a representation it cannot adapt.

Reading the tracks side by side is the point of this report. *What transfers* differs completely — Track I fine-tunes an entire learned model including its representation; Track II fine-tunes only a dynamics predictor over a representation that is shared and frozen — yet, as §6 shows, the two ends agree on the answers to both shared questions. That agreement, obtained from genuinely different machinery, is the strongest claim the combined study can make.

1.2 The commodity-GPU constraint

Every experiment here was built under one hard practical rule: it must run end to end on the hardware actually available to us (Google Colab — predominantly a free-tier T4; an A100 only occasionally), and the experiment *definition* (model size, data budgets, episode counts, seeds) must be identical across GPU tiers, with only throughput knobs (precision, dataloader workers, micro-batch or rollout-chunk size, wall-clock caps) differing. This constraint turned out to be

the technically interesting part of Track II: the stock DINO-WM loop is not merely slow on a T4 but structurally infeasible, projecting more than 30 hours per epoch regardless of batch size. Diagnosing and removing that cost *without altering the experiment* (§4.2) is what makes the Track II results obtainable at all, and the same discipline — pristine-upstream checkouts with reversible runtime patches, hardware-only profiles, and resumable stages — runs through both tracks (§6.3).

1.3 Contributions

1. **(Track I)** A faithful, budget-feasible R2-Dreamer transfer study on DMC vision (§3): a competent `walker_walk` world model gives a near-zero-shot head start on `walker_run` and a clean sample-efficiency advantage over scratch at a fixed A100 budget, together with a latent-geometry analysis showing the fine-tuned model lands in a distinct, better-structured representational basin.
2. **(Track II)** A latent-cache training path for DINO-WM that removes the dominant data-path cost while preserving the upstream training distribution for the selected windows ($\approx 21\times$ faster, §4.2); a resumable PointMaze reproduction; a low-data adaptation study (scratch vs. fine-tune) with both latent-prediction and real-environment planning metrics; and the observation that latent-prediction loss and planning success *dissociate* sharply (§4).
3. **(Track III)** A coupled local/global planning study on the trained DINO-WM model (§5): six planners spanning zero-order global search to first-order local optimization, with the result that a global re-score gate keeps first-order refinement *bounded-worsening* under the global model’s objective — recovering global-CEM quality where an ungarded hybrid collapses — while purely local planners are 100–1000 $\times$ cheaper per episode.
4. **(Synthesis)** A cross-track reading (§6) establishing that, at both ends of the representation spectrum, (a) fine-tuning dominates scratch at equal target-task budget, and (b) the scalar training metric understates a structural latent difference — basin geometry, plannability, or gradient reliability — that governs downstream control. We argue this is a concrete methodological warning for world-model evaluation.

1.4 A unifying thesis

Stated once, so the rest of the report can refer back to it: *across both ends of the representation spectrum, fine-tuning from a related-task checkpoint (Track I) or a full-data checkpoint (Track II) is a large sample-efficiency win at equal target-task budget in these single-seed runs, and the scalar quantity a world model is trained to minimize (episodic return, latent-prediction MSE, one-step surrogate error) is a weak proxy for the latent structure that a controller actually consumes.* The two tracks were designed and run independently; that they converge on the same two statements is the result.

2 Background and related work

Latent world models and imagination. The DreamerV3 lineage [3, 2, 1] learns a Recurrent State-Space Model (RSSM) [4] — a deterministic recurrent state plus a stochastic latent — and trains an actor-critic “in imagination” on rollouts of the learned dynamics rather than on environment frames [3]. Track I uses this backbone. PlaNet [4] plans directly in a learned latent; TD-MPC2 [11] pairs a learned latent model with sampling-based MPC.

Reconstruction-free representation objectives. Standard Dreamer learns its latent through pixel reconstruction with a decoder. A reconstruction-free family removes the decoder

and learns the representation through self-supervised auxiliary losses: DreamerPro [5] with prototypical representations, contrastive CPC/InfoNCE [7], and Barlow-Twins-style redundancy reduction [6]. R2-Dreamer (Track I) uses the Barlow objective on projected latents; a practical consequence is that the model has *no decoder* and therefore no imagined-frame visualization.

Frozen self-supervised features for control. The opposite design freezes a powerful pre-trained encoder and learns dynamics on top of its features. DINOv2 [9] provides such features; the JEPA world-model family (V-JEPA 2 [10]) and DINO-WM [8] plan over them. DINO-WM is the concrete, planning-focused instance reproduced in Tracks II–III: it encodes each frame to frozen DINOv2 patch tokens, trains a transformer predictor with a latent-consistency objective, and plans by sampling-based optimization of latent goal distance with no decoder in the loop.

Sampling-based and first-order planning. The cross-entropy method (CEM) [14] optimizes action sequences from forward rollouts only and is the planner reproduced in Tracks II–III. First-order (gradient-based) planners are sample-efficient per iteration but require a differentiable model; differentiable MPC [12] differentiates through the optimizer itself. The coupled local/global idea [13] resolves the accuracy-vs- differentiability tension by letting a large trusted model produce trajectories (never differentiated) while a lightweight surrogate supplies derivatives evaluated along those trajectories; Track III transplants this into DINO-WM latent planning.

We now present the two tracks in turn (§3, §4–§5), then read them together (§6). Notation common to the DINO-WM tracks is introduced in §4.

3 Track I — R2-Dreamer: transfer with a learned representation

Track I is a faithful, budget-feasible demonstration of the canonical Dreamer transfer experiment — train a latent world model on a source task, save and reload it, adapt it to a shifted task, and compare fine-tuning against scratch — using the *representation-regularized* R2-Dreamer objective on image observations. The three-way comparison is

```
source_base : walker_walk (from scratch),
target_finetune : walker_run (initialized from source_base),
target_scratch : walker_run (from scratch),
```

all on `dmc_vision` (64×64 RGB), orchestrated from a single Colab notebook against an unmodified upstream checkout.¹

3.1 Model

Backbone. A DreamerV3-style latent world model: a convolutional encoder maps each 64×64×3 frame to an embedding; an RSSM [4] carries a deterministic recurrent state and a stochastic latent and predicts forward dynamics; reward and continuation heads decode task quantities; and an actor–critic is trained entirely in imagination. The observation is vision only (no proprioceptive MLP input).

¹NM512/r2dreamer [18], a PyTorch reimplement of DreamerV3 [1] extending NM512/dreamerv3-torch [19]; behavioral changes are applied as marker-guarded, idempotent, reversible textual patches at launch time (§6.3).

Representation objective (the “R2”). The upstream code exposes a swappable representation loss (`model.rep_loss` $\in$ {`dreamer`, `r2dreamer`, `infonce`, `dreamerpro`}). We use `r2dreamer`: a reconstruction-free objective that learns the representation through a projector head trained with a redundancy-reduction (Barlow-Twins-style) loss on projected latents [6], placing it in the same reconstruction-free family as DreamerPro [5] and CPC/ InfoNCE [7]. The `r2dreamer` model has *no decoder*: there is no imagined-frame path, so visual analysis uses environment-rendered policy rollouts and the geometry of the RSSM latent states, not decoded reconstructions.

Parameter budget. The `size25M` configuration instantiates 26.53M trainable parameters, dominated by the RSSM and the R2 projector head (Table 1). Each checkpoint is 325.9 MB (fp32 weights plus Adam moments).

Component	Parameters	Role
RSSM	13,442,304	latent dynamics (recurrent + stochastic state)
Projector	5,898,240	R2 representation head (redundancy reduction)
Value (critic)	1,869,951	imagined-return value
Actor (policy)	1,776,396	imagination policy
Reward predictor	1,573,503	reward head
Continue predictor	1,475,713	episode-continuation head
Encoder (CNN)	493,824	$64 \times 64 \times 3 \rightarrow$ embedding
Total	26,529,931	

Table 1: R2-Dreamer components and parameter counts (read from the extraction logs of notebook 06).

3.2 Setup and resources

Source task `dmc_walker_walk`, target `dmc_walker_run`, both 64×64 RGB. The fine-tune run initializes *all* world-model weights from the source checkpoint with a strict state-dict load and does *not* carry over optimizer state (`pretrained_strict=true`, `load_optimizer=false`); `scratch` is identical except for random initialization. Both target runs share a 400k-step budget, so the comparison is at equal target experience (Table 2). Training and analysis ran on a single NVIDIA A100 (Colab), Python 3.11, with the DMC tasks on MuJoCo [17, 16]; image observations are produced by headless OSMesa rendering while the policy runs on CUDA.

Setting	Value	Setting	Value
Observation	<code>dmc_vision</code> $64 \times 64 \times 3$	Train ratio	224
Model	<code>size25M</code>	Parallel envs	8
Representation loss	<code>r2dreamer</code> (Barlow-style)	Eval cadence	every 50k steps, 5 episodes
Batch	16 sequences $\times$ length 64	Seed	0
Source steps	800k (reached ~ 796 k)	Target steps	400k (reached ~ 396 k)

Table 2: Track I configuration (the `a100_r2_vision25m` three-way preset).

3.3 Results

Source task is effectively solved. The source model reaches evaluation return 968.3 on `walker_walk` (the DMC walk reward saturates near 1000), so the checkpoint we fine-tune from is a competent walker — the prerequisite for a meaningful transfer test. Figure 2 shows the monotone climb to that ceiling.

Run	Init	Task	Final eval	Final train	Env steps
source_base	random	walker_walk	968.3	964.2	~796k
target_finetune	source_base	walker_run	742.3	693.4	~396k
target_scratch	random	walker_run	563.1	529.7	~396k

Table 3: Final results (DMC episodic return, 0–1000). “Eval” is the deterministic 5-episode score; “train” the rolling training return. For all three runs the best eval equals the final eval (no late-training collapse).

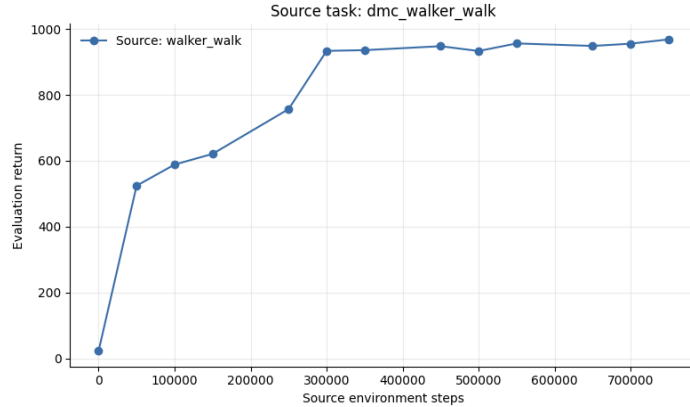


Figure 2: Source-task evaluation return over ~796k steps: **walker_walk** converges to ≈ 968 , providing the fine-tune initialization.

Transfer dominates scratch at every budget. This is the central Track I result (Figure 3). Three features stand out. (i) *A large warm start*: at the first evaluation, before any run-specific gradient steps, the fine-tuned policy already returns ~ 375 on **walker_run** versus ~ 25 for random scratch — walk and run share enough locomotion structure to transfer almost zero-shot. (ii) *Faster convergence*: by 100k target steps the fine-tune is at ~ 600 while scratch is near ~ 150 , a $\sim 4\times$ gap; the fine-tune passes 700 by ~ 300 k. (iii) *A persistent but shrinking lead*: the fine-tune ends at 742 and scratch at 563, a $+179$ ($+32\%$) advantage at the shared 400k budget. The advantage curve starts at ~ 352 , peaks at ~ 448 around 100k steps, and declines steadily as the scratch baseline — still rising at the budget limit — closes part of the gap. The result therefore demonstrates *sample efficiency*, not necessarily a higher asymptotic ceiling (§8).

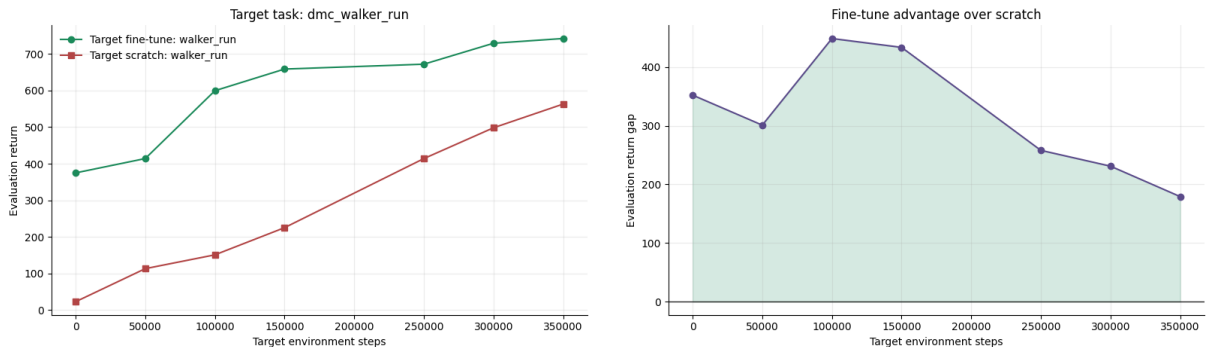


Figure 3: Target-task transfer. **Left:** evaluation return on **walker_run**, fine-tune (green) vs. scratch (red); fine-tune leads at every evaluation. **Right:** step-aligned advantage (fine-tune – scratch), peaking at ~ 448 near 100k steps and declining to $+179$ at the budget limit.

Training dynamics and optimization health. The training-episode curves (Figure 4, left) reveal a brief adaptation transient: the fine-tune begins near ~ 400 (inherited walk competence), *dips* to ~ 270 as the gait is repurposed for running — a short, recoverable negative-transfer phase — then climbs steadily past scratch. Optimization is stable for all three runs (Figure 4, right): every loss component descends and stays bounded, and the R2-specific Barlow redundancy-reduction term drops sharply and flattens, indicating the reconstruction-free objective trained as intended.

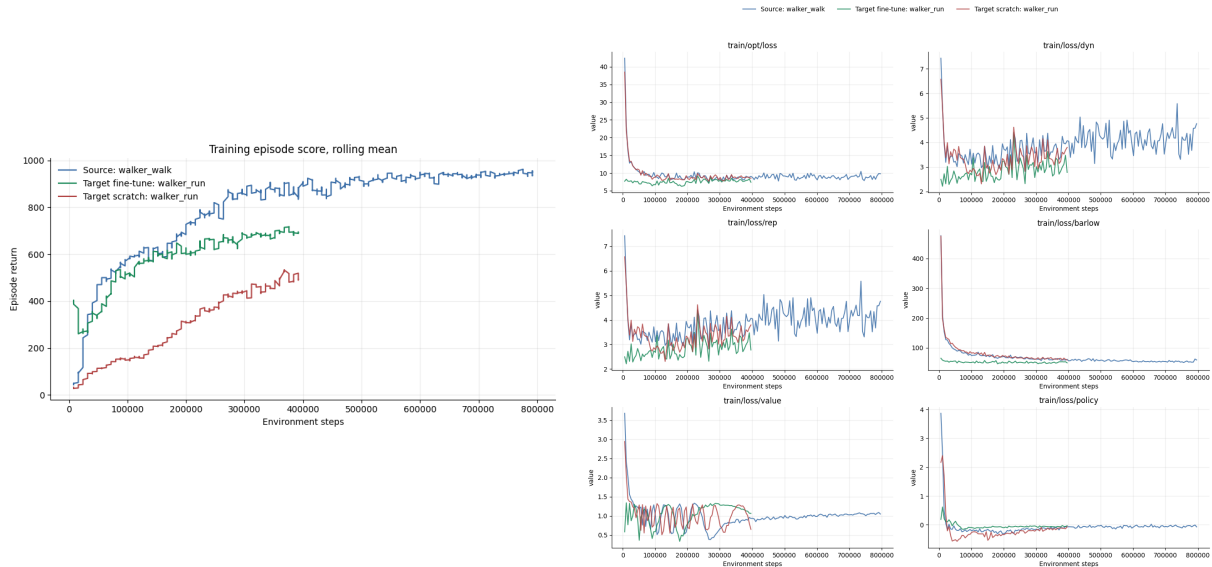


Figure 4: **Left:** rolling-mean training episode return (window 8): source (blue) to ~ 950 ; fine-tune (green) starts high, dips during gait re-adaptation, converges near ~ 690 ; scratch (red) rises from near zero to ~ 530 , still increasing. **Right:** optimization diagnostics — all loss components bounded and convergent, including `train/loss/barlow`.

Latent geometry: the two policies live in different basins. We extracted RSSM recurrent features along three evaluation episodes (1500 states each) for the fine-tune and scratch `walker_run` policies and projected them with a shared-basis PCA (Figure 5). Although both solve the *same* task, their latent states occupy clearly separated regions — near-linearly separable along PC1 — so the warm-started model encodes the task *differently*, not merely faster. The 3D view is more striking: the fine-tuned model’s latents trace a large, organized ring/limit-cycle manifold consistent with a periodic running gait, whereas the scratch model’s latents collapse into a small, compact cluster, mirroring its lower return. The top three components explain only $\approx 9.5/3.8/3.5\%$ of the variance ($\sim 17\%$ total), so these projections capture the dominant linear structure, not the full latent geometry (§8). This geometric difference, which the scalar return only partly conveys, is the first instance of the report’s second thesis (§6).

The results notebook (06) also renders one environment-camera rollout per checkpoint plus a rotating 3D latent-PCA animation; because the model is decoder-free these are true renderings of the executed policy, and their qualitative content tracks Table 3 (a near-solved walk, a competent fine-tune run, a weaker scratch run).

4 Track II — DINO-WM: planning with a frozen representation

Track II reproduces DINO-WM [8] on PointMaze under the commodity-GPU constraint of §1.2, and uses it for a low-data adaptation study that mirrors Track I’s question at the frozen-representation end of the spectrum.

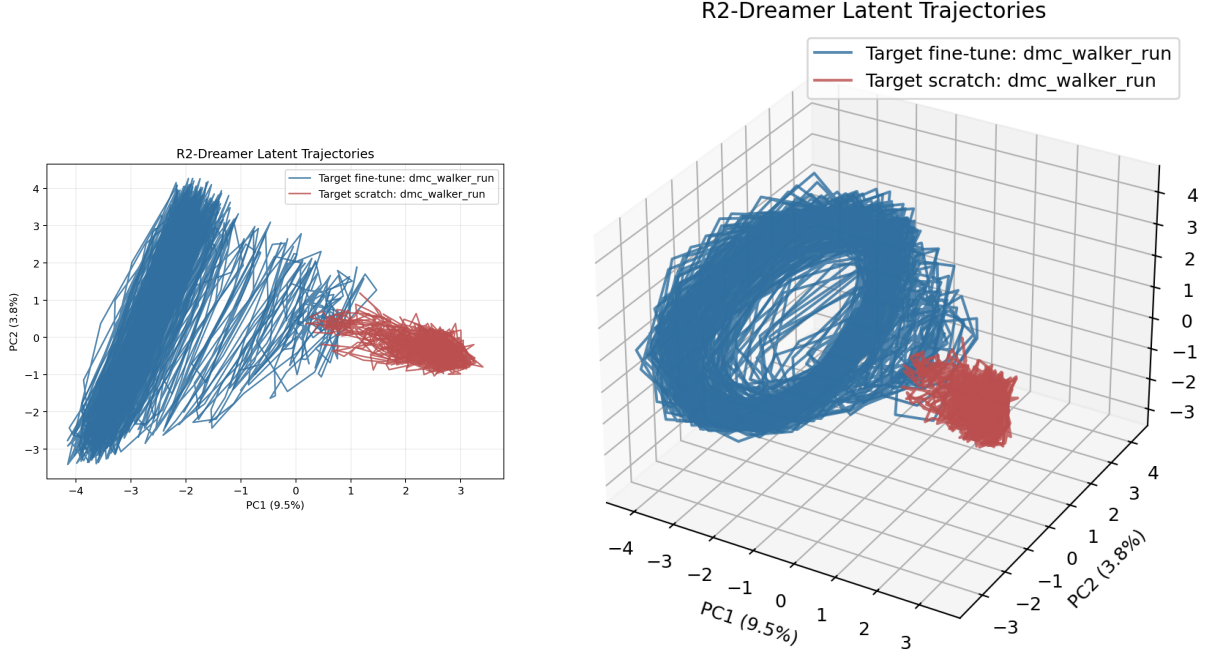


Figure 5: Shared-basis PCA of RSSM latent trajectories, fine-tune (blue) vs. scratch (red) on `walker_run`. **Left (2D)**: separation along PC1. **Right (3D)**: the fine-tune traces a large cyclically structured manifold; scratch occupies a compact blob.

Notation. PointMaze episodes are frame sequences with raw 2-D actions $\mathbf{a}_t \in [-1, 1]^2$ and proprioception. Following DINO-WM, one *model step* advances f raw steps (frameskip $f=5$); the model-step action is the folded block $\bar{\mathbf{a}}_k = [\mathbf{a}_{kf}; \dots; \mathbf{a}_{(k+1)f-1}] \in \mathbb{R}^{fA}$. Each frame is encoded to $\mathbf{z} \in \mathbb{R}^{P \times D}$ with $P=196$ patches and $D=384$ (DINOv2 ViT-S/14 at 224 px, a 14×14 patch grid). The predictor consumes $H=3$ context frames and predicts the next latent (`num_pred=1`). Episode-level train/validation splits are drawn from a seeded permutation of the *uncapped* episode count, so split membership is invariant to how many rollouts a run uses.

4.1 The DINO-WM model as reproduced

Encoder (frozen). `dinov2_vits14`, feature key `x_norm_patchtokens`, frozen throughout (`train_encoder=false`), with the upstream preprocessing transform. **Predictor:** a ViT (`models.vit.ViTPredictor`; depth 6, 16 heads, MLP width 2048, dropout 0.1) over the flattened $H \times P = 3 \times 196 = 588$ token sequence. Actions and proprioception are embedded (10+10) and concatenated along the feature axis (`concat_dim=1`), giving working dimension $D+10+10 = 404$; a causal temporal mask is applied. **Objective:** with $\hat{\mathbf{z}}$ the predicted next latent and $\mathbf{z}$ the detached target,

$$\mathcal{L} = \|\hat{\mathbf{z}} - \text{sg}[\mathbf{z}]\|_2^2, \quad (1)$$

the latent-consistency MSE, reported per epoch as a training and a validation (“h-step”) component. The runs are *decoder-free* (`has_decoder=false`): no pixel reconstruction is trained, which halves memory and matches the planning objective (planning never decodes).

4.2 Feasible training: the latent-cache path

Why the stock loop costs >30 h/epoch. Profiling identified three compounding causes, none addressable by batch size (confirmed: batch 4 and batch 32 both project $\sim 19\text{--}21$ h/epoch on a T4). (1) *Per-sample episode reload*: upstream `TrajSlicerDataset` fetches a 4-frame window by `torch.load`-ing the *entire* episode (~ 93 frames) and transforming every frame, then discarding

all but four; at $\sim 147k$ windows/epoch this re-reads $\sim 1600\times$ the dataset per epoch through the Drive FUSE mount. (2) *Online re-encoding of frozen features*: with `train_encoder=false` the DINOv2 outputs are dataset constants, yet every frame of every window is re-encoded every epoch — pure recomputation. (3) *Operational confounders*: single-threaded loading and reads from Drive rather than local disk.

Encode once, reuse forever. Because the encoder is frozen, its outputs can be precomputed. Three marker-guarded patches, installed at launch: (1) *one-time precompute* — each episode is read once, preprocessed *exactly* as upstream, encoded under autocast, and written as a per-episode `fp16.npy` plus a manifest (~ 30 GB for all 2200 episodes, on session-local disk; idempotent and resumable); (2) *a drop-in latent dataset* selected by Hydra override that subclasses the upstream dataset, so actions, states, proprioception, normalization, and the seeded split are *bit-identical* to the online path by construction (the subclass overrides only the visual features; `scripts/dino_wm/validate_latent_cache.py` checks the split, manifest, latent shape, dtype, and planning-path dispatch), memory-mapping only the ~ 0.6 MB of latents each window needs instead of a ~ 13 MB episode; (3) *an input-dispatch bypass* in `encode_obs` (4-D latent inputs skip the encoder; 5-D image inputs take the original path), so planning, which encodes real observations, is untouched. For the selected windows this preserves the upstream split, actions, proprioception, normalization, and the planning-time image-encoding path exactly — the only thing that changes is when and how often the frozen (`fp16`-cached) encoder runs. The one deliberate departure is the full T4 run’s stride-2 window subsampling (Table 4), a throughput-biased lower-bound experiment rather than a paper-exact schedule (§8); the low-data pair uses the full stride-1 set.

Configuration	Steps/epoch	Step time (T4)	Epoch (T4)	Epoch (A100, est.)
Stock, batch 4	36,815	~ 3 s	> 30 h	—
Stock, batch 32	4,602	15.9–17 s	19–21 h (proj.)	—
Latent cache, batch 32, stride 1	4,602	~ 0.76 s	~ 58 min	6–12 min
Latent cache, batch 32, stride 2	2,330	0.72–0.76 s	31 min (meas.)	3–6 min

Table 4: Training throughput before and after the latent-cache path ($\approx 21\times$). The residual step cost is the depth-6 ViT predictor itself — the quantity the experiment is meant to measure. “Stride 2” subsamples window starts and was used only for the full run on T4; the low-data pair uses the full stride-1 set.

4.3 Planning and evaluation

Planning uses the upstream CEM [14]/MPC path unchanged. Given start and goal observations, CEM optimizes an open-loop action sequence to minimize the predicted latent distance to the goal encoding:

$$J(\bar{\mathbf{a}}_{1:H}) = \frac{1}{PD} \|\hat{\mathbf{z}}_H(\bar{\mathbf{a}}_{1:H}) - \mathbf{z}_{\text{goal}}\|_2^2, \quad (2)$$

iterating sample/score/refit (population 100, 10 iterations, goal horizon $G=5$, `goal_source=random_state`). Crucially, while CEM *optimizes* an imagined latent quantity, **success is measured in the real MuJoCo environment**: the planned actions are executed and success is the environment’s own goal-reaching test. Reported success rates are therefore real closed-loop control outcomes, not the model’s self-assessment.

4.4 Runs

Three runs enter the results (Table 5); all train the predictor only, on cached latents, with identical seeds, normalization, and splits. Precision is `bf16` on A100/L4 and `fp16` on T4; all runs are seed 0. CEM population/iters 100/10, 200 planning evaluations per checkpoint.

Run	Data (train+val)	Epochs	Initialization	Planning evals
Full no-decoder	2000 + 200	10	random	200
Low-data scratch	300 + 60	30	random	200
Low-data fine-tune	300 + 60	20	full checkpoint	200

Table 5: Runs. Fine-tuning loads the predictor and action/proprio encoders from the full run’s checkpoint and continues at reduced learning rates ($10^{-5}/10^{-4}$); the frozen DINOv2 encoder is shared. The full run was trained on a T4 with stride-2 window subsampling (Table 4).

4.5 Results

All numbers are read from the *latest final_eval* row of each run’s planner `logs.json`, at 200 evaluation episodes per checkpoint (Wilson half-width ≈ 0.06).

Training loss: fine-tuning starts where scratch ends. The fine-tuned predictor attains at *epoch 1* the validation loss the scratch baseline reaches only after ~ 30 epochs (Figure 6): pre-training delivers the converged predictor for $\sim 30\times$ fewer updates on the low-data task. The honest caveat is that scratch *does* converge to the same order of magnitude by epoch 30 (Figure 6: ≈ 0.021 vs ≈ 0.012) — which makes the planning result that follows all the more pointed.

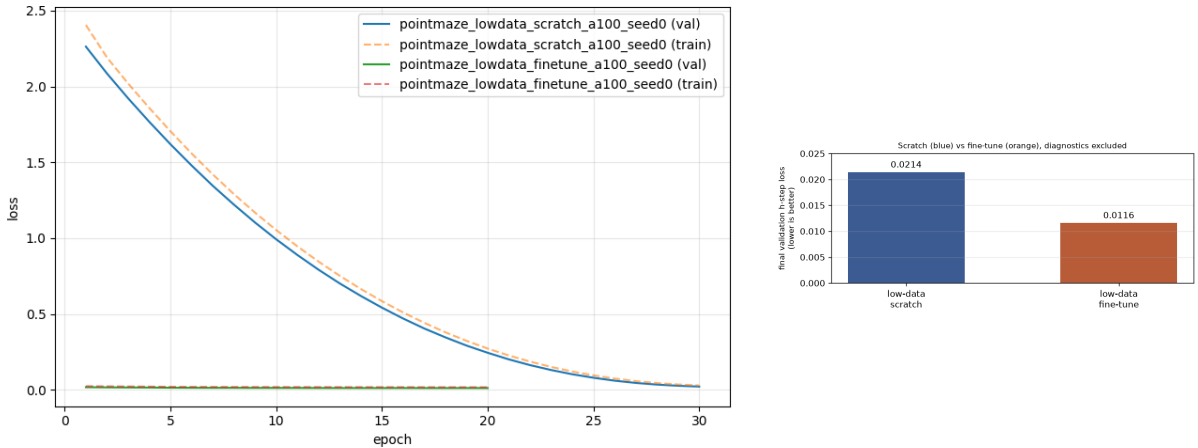


Figure 6: **Left:** per-epoch training (dashed) and validation (solid) latent-prediction loss. Scratch descends from ≈ 2.3 to ≈ 0.02 over 30 epochs; the fine-tune begins at ≈ 0.015 at epoch 1 — already the scratch run’s converged level — and stays there. **Right:** final validation h-step loss, scratch vs. fine-tune (≈ 0.021 vs ≈ 0.012 , same order of magnitude).

Planning success in the real environment. At $n=200$ the Wilson intervals tighten to ± 0.06 (Figure 7). Two clean results stand out. **Fine-tune** $>$ **scratch** is decisive (0.275 vs 0.10; an episode-level two-proportion $z=4.5$, $p < 10^{-5}$ — a Bernoulli approximation over the 200 evaluation episodes, not a run-level test, since all runs are seed 0): pre-training on the full corpus and adapting on 300 rollouts yields $2.75\times$ the planning success of training on those same 300 rollouts from scratch. **Fine-tune vs. full-data** (0.275 vs 0.23) is statistically indistinguishable ($p=0.30$): a full-data-pretrained predictor, adapted on 300 target rollouts, reaches a similar 200-episode planning score to the full-data checkpoint under this evaluation. The full model’s 0.23 is moreover a lower bound (T4 stride-2 subsampling, half the gradient updates of the paper-exact stride-1 set); an A100 stride-1 run is needed to test whether the full-data score improves under the paper-exact window schedule. (The larger sample moved all three estimates down from the noisier 50-eval interim, $0.34/0.26/0.12 \rightarrow 0.275/0.23/0.10$, without disturbing the ranking.)

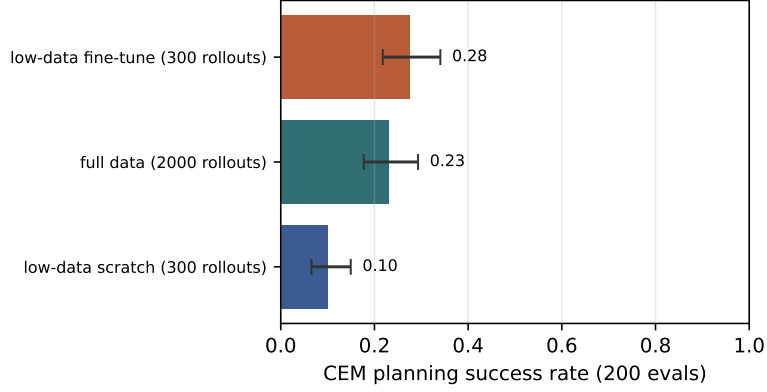


Figure 7: Closed-loop CEM planning success in MuJoCo PointMaze, 200 evaluation episodes per checkpoint, 95% Wilson intervals: fine-tune 0.275, full-data 0.23, scratch 0.10.

The loss/planning dissociation. The central empirical finding of Track II is the contrast between Figure 6 and Figure 7: scratch and fine-tune converge to the same order of magnitude in validation latent-prediction loss (≈ 0.021 vs ≈ 0.012) yet differ $2.75\times$ in real-environment planning success (0.10 vs 0.275 , $p < 10^{-5}$). The scalar loss alone does not rank downstream control quality: this $\sim 2\times$ residual loss gap (same order of magnitude) coincides with a near-tripling of planning success, which points to additional geometric or calibration differences that the scalar MSE does not capture — the fine-tuned model’s latent geometry appears more *plannable* (smoother or better-calibrated for CEM) despite a similar average prediction error. The practical consequence is direct: *a planning metric must be carried alongside the loss curve*; the loss alone would have reported these two models as equivalent. This is the second instance of the report’s second thesis (§6).

Planner diagnostics: CEM is genuinely optimizing. The per-iteration telemetry (Figure 8) confirms CEM is exploiting the learned dynamics rather than stumbling onto goals: environment success of the current best plan rises and mean state distance falls monotonically over the 10 iterations for every trained model, with scratch improving least. The embedding-divergence panel shows the better models keep imagination closest to executed reality at the end of optimization — exactly the property a planner needs.

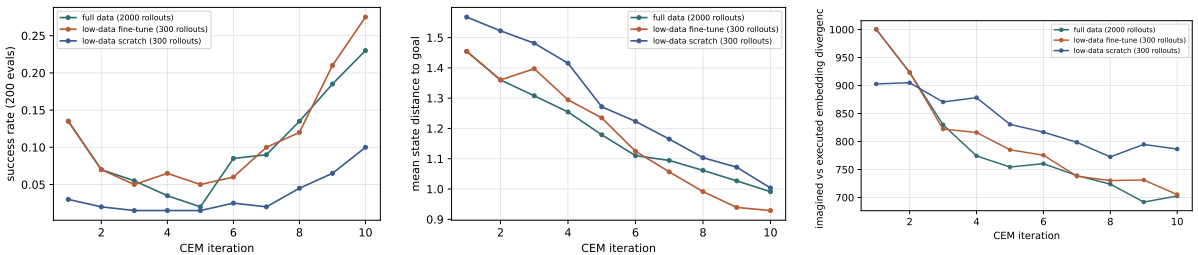


Figure 8: Planner-internal telemetry per CEM iteration (exact values from `logs.json`). **Left:** environment success of the current best plan rises across the 10 iterations (full $0.135 \rightarrow 0.23$; fine-tune $0.135 \rightarrow 0.275$; scratch $0.03 \rightarrow 0.10$). **Middle:** mean state distance to goal falls correspondingly (fine-tune $1.45 \rightarrow 0.93$). **Right:** imagined-vs-executed embedding divergence — the model’s own rollout error; the fine-tune and full models keep imagination closest to reality.

5 Track III — Coupled local/global planning on the DINO-WM model

The trained DINO-WM checkpoint (§4) is the *global* model for a second study: cheap first-order planning against a trusted, non-differentiated world model [13]. Zero-order planners (CEM) work with any accurate forward model but scale poorly in horizon and action dimension; first-order planners are sample-efficient per iteration but need a differentiable model — and a model small enough to differentiate cheaply is usually inaccurate enough to be *exploited*, with gradient descent walking into regions where it is wrong. The coupled recipe decouples the two roles: the large model produces trajectories and is never differentiated, while a lightweight *local* surrogate supplies derivatives evaluated *along the global model’s trajectories*. We instantiate it on the PointMaze DINO-WM model and ask whether first-order refinement through a cheap surrogate can recover global-CEM-quality plans at a fraction of the global-model compute.

5.1 Method

The **global model** is the frozen DINO-WM latent predictor, used as forward simulator and scorer under `no_grad`. The **local model** is a small residual-MLP surrogate $\mathbf{x}_{k+1} = \mathbf{x}_k + g_\theta([\mathbf{x}_k; \bar{\mathbf{a}}_k])$ (width 512, 3 layers) over a *frozen* orthogonal projection $\mathbf{x} = W_p \text{pool}(\text{LN}(\mathbf{z})) \in \mathbb{R}^{256}$ of the same latents, differentiable in the actions and trained on cached transitions. Freezing the projection is principled: a jointly trained projector admits the degenerate optimum $W_p=0$ (representation collapse), whereas a frozen near-orthogonal projection of LayerNormed features preserves distances (Johnson–Lindenstrauss) and removes the collapse mode. Six **planners** are compared: `global_cem` (zero-order on the global model); `local_cem`, `local_gd`, `local_adam` (zero/first-order on the surrogate alone, isolating model error from optimizer effects); `hybrid_cem_local_refine` (global CEM, then local first-order refinement of the winning sequence); and `hybrid_cem_local_refine_global_rescore` (the same, but the global model re-scores the refinement and rejects it only if it *worsens* the global objective by more than a relative tolerance $\tau=0.05$ — a bounded-worsening gate, so a refinement up to τ worse is still accepted). Three safeguards against surrogate exploitation are built in — the frozen projection (anti-collapse), an action-smoothness penalty, and the global re-score gate — and the local–global *disagreement* (surrogate prediction vs. the global model’s executed step) is logged throughout.

5.2 Protocol and its stated bias

Planners are scored on offline latent goal reaching over 100 held-out episodes: normalized final distance $\rho = \|\mathbf{z}_{\text{fin}} - \mathbf{z}_{\text{goal}}\|^2 / \|\mathbf{z}_{\text{start}} - \mathbf{z}_{\text{goal}}\|^2$ and success = $\mathbb{1}[\rho < 0.5]$. **Because the global model is both simulator and judge**, `global_cem` is structurally favored on distance — it optimizes exactly the quantity it is scored on — so the distance/success columns establish only that the other planners are *competent*, while the efficiency columns (wall time, forward/backward call counts) and the acceptance rate carry the comparative claim. A *reference replay* of the dataset’s true action sequence through the same simulator calibrates the global model’s own rollout error at $\rho_{\text{ref}} = 0.058$.

5.3 Results

The surrogate reaches a final validation one-step loss of 0.0058 and a K -step rollout MSE of 0.0029 at 20,000 steps (Figure 9). The six-planner comparison is Table 6.

Findings.

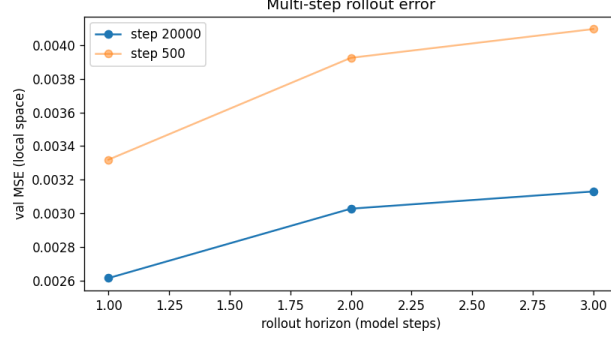


Figure 9: Surrogate validation rollout error vs. horizon (PointMaze, seed 0), exported by notebook 08.

Planner	succ.	ρ	wall/ep. (s)	glob. fwd	loc. fwd	bwd	accept	disagr.
global_cem	0.97	0.138	231.6	1.50M	—	—	—	—
local_cem	0.45	0.675	0.22	—	1.50M	—	—	0.0020
local_gd	0.13	1.306	2.60	—	50k	50k	—	0.0028
local_adam	0.41	0.699	2.67	—	50k	50k	—	0.0019
hybrid_cem_local_refine	0.54	0.632	234.1	1.50M	50k	50k	1.00	0.0019
hybrid..._global_rescore	0.97	0.138	234.2	1.50M	50k	50k	0.028	0.0024
reference replay	—	0.058	—	—	—	—	—	—

Table 6: Planner comparison on offline latent goal reaching, 100 held-out PointMaze episodes (surrogate at 20k steps). Distances are judged by the global model (see the stated bias), so the efficiency columns carry the comparative claim. **accept**=refinement acceptance rate; **disagr.**=mean local-global disagreement.

1. **Global CEM tops the distance metric** (0.97, $\rho=0.138$), as the judge bias predicts; even so its 0.138 is $\approx 2.4\times$ the reference-replay floor (0.058) — the global model does not perfectly close its own gap.
2. **Pure local planners are far cheaper but weaker** on the global-judged metric: `local_cem` 0.45 at 0.22s/episode, `local_adam` 0.41 at 2.7s, `local_gd` 0.13 — against `global_cem`’s 231s/episode. The surrogate forward is $\sim 1000\times$ cheaper per call, but at this capacity it cannot plan to global quality alone.
3. **The unguarded hybrid hurts.** `hybrid_cem_local_refine` accepts *every* refinement (accept=1.00) and falls to 0.54: refining on the surrogate walks a good global plan into regions where the surrogate is wrong — the exploitation failure mode the safeguards target.
4. **The global re-score gate fixes it.** `hybrid..._global_rescore` rejects 97.2% of refinements (accept=0.028) and recovers global-CEM quality *exactly* (0.97, $\rho=0.138$). This is the headline: the gate turns large surrogate-exploitation failures into no-ops, so the hybrid stays bounded-worsening under the global model’s objective (within $\tau=0.05$ of the CEM it began from) and here matches its quality exactly.
5. **Disagreement is small, yet gradients are unreliable.** The local-global disagreement stays ≈ 0.002 — the surrogate tracks the global model’s one-step behavior — yet its gradient *directions* improve the global objective rarely enough that the gate accepts only 2.8% of them. One-step tracking accuracy and gradient usefulness are not the same thing — the third instance of the report’s second thesis (§6).

Synthetic sanity check. On the exact synthetic point-mass task (closed-form perfect global model; a few hundred surrogate steps; a correctness signal rather than statistics) the expected

ordering holds: all six planners succeed, `local_adam` attains the lowest final distance, the hybrids improve on plain CEM with their refinements accepted, and `local_gd` is weakest. In an adversarial unit test where the surrogate’s action gain is sign-flipped, refinement reliably worsens the global cost and the re-score gate rejects it, returning the CEM sequence unchanged.

What this establishes. The protocol measures planner behavior *under the global model’s belief*. It shows first-order refinement through a cheap surrogate can be made *bounded-worsening* under the global model’s objective — rejected whenever it worsens that objective by more than $\tau=0.05$ — by a global re-score gate, and that the pure local planners are 100–1000 $\times$ cheaper per episode. It cannot, by construction, rank planners against reality; real-environment confirmation in MuJoCo PointMaze is the headline next step (§9).

6 Cross-track synthesis

The two tracks were designed and run independently and instantiate genuinely different machinery — a learned, reconstruction-free latent versus a frozen, pretrained one; locomotion control versus goal-reaching planning. Read together, they converge on the two statements of §1.4.

6.1 Fine-tuning wins at both ends of the representation spectrum

At the *learned-representation* end, fine-tuning the whole R2-Dreamer model from `walker_walk` gives an almost zero-shot head start on `walker_run` (~ 375 vs ~ 25), a $\sim 4\times$ mid-training lead, and a +179 (+32%) advantage at a fixed 400k budget. At the *frozen-representation* end, fine-tuning only the DINO-WM dynamics predictor from a full-data checkpoint converges in a single epoch ($\sim 30\times$ fewer updates) and yields $2.75\times$ the scratch baseline’s real-environment planning success (0.275 vs 0.10), reaching a planning score similar to the full-data checkpoint. *What* transfers is entirely different — an entire model including its representation, versus a dynamics head over a shared frozen latent — but the qualitative answer is the same: at equal target-task budget, warm-starting (from a related task in Track I, from a full-data checkpoint in Track II) is a large sample-efficiency win, and in both tracks the scratch baseline had not closed the gap within the budget studied.

6.2 The scalar fit is not the usable structure

The sharper shared finding is that the scalar quantity each model is trained to minimize understates a *structural* difference in the latent that governs downstream behavior (Table 7). In Track I, the +32% return gap accompanies a *qualitative* representational difference the scalar does not convey: two policies solving the same task occupy distinct, near-linearly- separable PCA basins, and the better one carries a limit-cycle manifold where the worse one collapses to a blob — fine-tuning changed *what* was learned, not just how fast. In Track II, two predictors with latent-prediction MSE of the *same order of magnitude* differ $2.75\times$ in planning success: similar average prediction error, unequal plannability. In Track III, a surrogate whose one-step disagreement with the global model is a mere ≈ 0.002 produces gradient directions reliable enough to survive the re-score gate only 2.8% of the time: one-step fit does not imply usable gradients. In each case a scalar fit/accuracy metric (return, latent-MSE, one-step error) is a weak proxy for the latent or gradient *geometry* a controller consumes — which is why every track had to carry a structural or downstream diagnostic (PCA geometry, a real- environment planning metric, refinement acceptance) alongside the headline scalar.

Track	Scalar signal (mis-leading)	What it hides	Diagnostic that reveals it
I R2-Dreamer (learned latent)	episodic return; both policies solve <code>walker_run</code>	the two policies encode the task in <i>different latent basins</i>	shared-basis PCA: PC1-separable; limit-cycle manifold vs. collapsed blob
II DINO-WM (frozen latent)	validation latent-MSE ≈ 0.012 vs ≈ 0.021 (same order)	a $2.75\times$ gap in real-environment <i>plannability</i>	closed-loop CEM success in MuJoCo (0.275 vs 0.10, $p < 10^{-5}$)
III local/global (surrogate)	one-step local-global disagreement ≈ 0.002	<i>gradient directions</i> that rarely improve the global objective	global re-score acceptance rate = 2.8%

Table 7: One finding, three instances: across the spectrum, the scalar a model is trained to fit understates a structural latent/gradient difference that controls downstream behavior, and only a structural or downstream diagnostic exposes it.

6.3 A shared engineering methodology

Both tracks reach these results through the same discipline, which is itself part of the contribution because it is what made the studies feasible and trustworthy on free hardware. (1) *Pristine upstream, runtime patches*: all behavioral changes (R2-Dreamer checkpoint load/save and headless-render guard; DINO-WM latent cache, validation `no_grad`, fine-tune init, decoder-free video recording) are marker-guarded, idempotent, reversible textual patches applied at launch with backups, unit-tested against the real upstream source; neither upstream repository is vendored or edited in place. (2) *One experiment, hardware-only profiles*: GPU-tier configs differ only in throughput knobs (precision, dataloader workers, wall-clock caps, rollout-chunk or window stride); model sizes, data budgets, episode counts, and run names are shared, so a run started on one tier resumes on another. (3) *Resumability everywhere*: per-epoch plus rolling intra-epoch checkpoints, and per-episode planning resume (the latest `final_eval` is authoritative), bound any disconnection to minutes of lost work — essential when a single sweep (e.g. the ~ 20 h six-planner evaluation) exceeds a free-tier session. (4) *Smoke + unit tests*: a ~ 10 –15 min end-to-end smoke exercises the exact production pipeline before any costly run, and ~ 95 (DINO-WM) / ~ 170 (local/global) tests cover patch composition, command rendering, run-state gates, metrics extraction, latent-dataset parity, and analytic planner correctness (including a sign-flipped surrogate that asserts the rejection gate fires).

7 Computational resources

Every experiment fits inside free-tier Colab limits, with the experiment *definition* identical across GPU tiers (§1.2); only throughput knobs differ between a T4, an L4, and an A100. Baseline DINO-WM runs use a free-tier T4 (16 GB, fp16); an A100-40 GB or L4 (bf16) is used opportunistically. The enabling optimization is the latent cache: the stock loop re-encodes $\sim 1600\times$ the dataset through frozen DINOv2 every epoch, projecting 19–30 h/epoch on a T4; precomputing the patch latents once cuts per-step cost $\approx 21\times$ (Table 4), to 31 min/epoch on a T4 at stride 2 or 3–6 min on an A100. Track I’s R2-Dreamer runs use a single A100 at a fixed 400k-step target budget per run.

8 Threats to validity and limitations

- **Single seed throughout.** Every run is seed 0; there are no error bars. The Track I mid-training $\sim 4\times$ gap and +179 final advantage, and the Track II $2.75\times$ planning gap ($z=4.5$, an

Stage	Budget	Wall-clock
<i>Track II DINO-WM (PointMaze)</i>		
Latent precompute (one-time)	2,200 episodes $\rightarrow$ DINOv2	amortized; ~ 30 GB cache
Predictor training, full	10 epochs, 2,000+200 traj.	~ 31 min/epoch T4; 3–6 min A100
Predictor training, low-data ($\times 2$)	20–30 ep, 300+60 traj.	well under the full run (15% data)
Planning eval (per checkpoint)	200 episodes, CEM 100×10	~ 47 min (A100, measured)
<i>Track III local/global</i>		
Surrogate training	20,000 steps	~ 1 – 2.5 h (T4)
Six-planner eval	100 episodes $\times$ 6 planners	~ 20 h total (resumable)
<i>Track I R2-Dreamer (DMC vision)</i>		
Source / target training	800k / 400k steps, size25M	single A100, per run

Table 8: Wall-clock footprint on the stated hardware. The six-planner sweep is dominated by the three planners that call the global model (≈ 232 – 234 s/episode, ≈ 6.5 h per 100 episodes) against ≈ 0.2 – 2.7 s for the purely local planners — the $\sim 1000\times$ asymmetry that motivates the coupled design.

episode-level Bernoulli approximation over the 200 evaluation episodes), are large relative to typical run-to-run noise, but run-level statistical claims would require additional seeds.

- **Fixed budgets; scratch not converged.** The R2-Dreamer scratch baseline is still rising at 400k steps, so its +179 deficit measures sample efficiency at a fixed budget, not an asymptotic ceiling. Likewise the DINO-WM full model’s 0.23 is a lower bound (T4 stride-2 subsampling).
- **Loss/planning and PCA caveats.** The Track I PCA explains only $\sim 17\%$ of variance in three components, so the basin separation and ring structure are real on the dominant axes but not the full latent geometry. The Track II planning estimates are 200-episode point estimates (± 0.06 Wilson).
- **Offline local/global protocol.** Track III judges distances with the global model itself, so it ranks planners *under the model’s belief*, not against reality; the efficiency columns and acceptance rate carry the comparative claim, and real-environment confirmation is future work.
- **No representation-loss ablation (Track I).** We run only `rep_loss=r2dreamer` and do not compare against `dreamer`, `infonce`, or `dreamerpro`, so the report characterizes *this* configuration rather than attributing the transfer result to the R2 objective specifically. Being decoder-free, it also offers no reconstruction/imagination metric.
- **Precision across tiers.** T4 runs use fp16, A100/L4 bf16/TF32; cross-tier results are statistically equivalent, not bit-identical.

9 Conclusion and future work

Across two world-model designs that sit at opposite ends of the representation spectrum — a learned, reconstruction-free R2-Dreamer latent and a frozen, pretrained DINO-WM latent — and under a strict commodity-GPU budget, the same two conclusions hold. Fine-tuning from a related-task checkpoint (Track I) or a full-data checkpoint (Track II) is a large single-seed sample-efficiency win over training from scratch at equal target-task budget in the reported fixed-budget runs; and the scalar metric a world model is trained to minimize is a weak proxy for the latent structure a controller consumes, so world-model evaluation must carry a structural or downstream diagnostic alongside the loss or return. On the planner side, a global re-score gate keeps cheap first-order refinement bounded-worsening under the global model’s scored objective

— rejecting refinements that worsen it by more than a small tolerance — while keeping the door open to its 100–1000× per-call efficiency.

The most valuable next steps are shared across tracks: (1) *seed replication* of every headline comparison (the Track I target pair, the Track II low-data pair, the Track III sweep) to put error bars on the advantages; (2) *extended/parity budgets* — a longer R2-Dreamer scratch run to separate “head start” from “higher ceiling,” and an A100 stride-1 full-data DINO-WM training to replace the stride-2 lower bound; (3) *real-environment confirmation* of the local/global planner ranking in MuJoCo PointMaze, the one claim the offline protocol cannot make; (4) *ablations* — the R2-Dreamer representation-loss family, and the local/global projection/context/optimizer levers and FOG-style online surrogate fine-tuning motivated by the low acceptance rate; and (5) *harder shifts* — Meta-World or proprio→vision transfer for R2-Dreamer, and PushT (blocked upstream on a video-backed latent cache) for the DINO-WM tracks, where contact dynamics should stress both the surrogate and the re-score safeguard most.

Reproducibility

Code lives in the `wm-prediction` repository. **Track I:** `scripts/r2dreamer`, `configs/r2dreamer`; notebook 01 trains (Colab A100, Python 3.11, default `a100_vision` preset) and notebook 06 regenerates results, rollout videos, and the latent analysis. **Tracks II–III:** `src/wm_poc/dino_wm` and `src/wm_poc/local_global`, `scripts/dino_wm` and `scripts/local_global`, `configs/dino_wm` and `configs/local_global`; notebook 02 runs the DINO-WM pipeline and notebook 03 the local/global pipeline (both self-gating and resumable, T4 or A100), with notebooks 07/08 rendering the results. The latent-cache design is documented in `docs/dino_wm_latent_cache_training.md` and the local/global method in `docs/local_global_techniques.md`; planning figures are produced from recorded telemetry (`reports/pointmaze_planning_logs_200evals.csv`) by `reports/make_planning_figures.py`. Every figure and number in this report is read from the committed outputs of notebooks 06/07/08.

References

- [1] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. *Mastering Diverse Domains through World Models*. arXiv:2301.04104, 2023.
- [2] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. *Mastering Atari with Discrete World Models*. ICLR, 2021. arXiv:2010.02193.
- [3] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. *Dream to Control: Learning Behaviors by Latent Imagination*. ICLR, 2020. arXiv:1912.01603.
- [4] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. *Learning Latent Dynamics for Planning from Pixels*. ICML, 2019. arXiv:1811.04551.
- [5] F. Deng, I. Jang, and S. Ahn. *DreamerPro: Reconstruction-Free Model-Based Reinforcement Learning with Prototypical Representations*. ICML, 2022. arXiv:2110.14565.
- [6] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny. *Barlow Twins: Self-Supervised Learning via Redundancy Reduction*. ICML, 2021. arXiv:2103.03230.
- [7] A. van den Oord, Y. Li, and O. Vinyals. *Representation Learning with Contrastive Predictive Coding*. arXiv:1807.03748, 2018.
- [8] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. *DINO-WM: World Models on Pre-trained Visual Features Enable Zero-shot Planning*. ICML, 2025. arXiv:2411.04983.

- [9] M. Oquab, T. Darcet, T. Moutakanni, et al. *DINOv2: Learning Robust Visual Features without Supervision*. Transactions on Machine Learning Research (TMLR), 2024. arXiv:2304.07193.
- [10] M. Assran, A. Bardes, D. Fan, Q. Garrido, et al. *V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning*. arXiv:2506.09985, 2025.
- [11] N. Hansen, H. Su, and X. Wang. *TD-MPC2: Scalable, Robust World Models for Continuous Control*. ICLR, 2024.
- [12] B. Amos, I. D. J. Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter. *Differentiable MPC for End-to-end Planning and Control*. NeurIPS, 2018.
- [13] J. Amigo, R. Khorrambakht, N. Mansard, and L. Righetti. *Coupled Local and Global World Models for Efficient First-Order RL*. arXiv:2602.06219, 2026.
- [14] P.-T. de Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein. *A Tutorial on the Cross-Entropy Method*. Annals of Operations Research, 134(1):19–67, 2005.
- [15] Y. Tassa et al. *DeepMind Control Suite*. arXiv:1801.00690, 2018.
- [16] S. Tunyasuvunakool et al. *dm_control: Software and Tasks for Continuous Control*. Software Impacts, 6:100022, 2020.
- [17] E. Todorov, T. Erez, and Y. Tassa. *MuJoCo: A Physics Engine for Model-Based Control*. IROS, 2012.
- [18] N. Morihira, A. Nahar, K. Bharadwaj, Y. Kato, A. Hayashi, and T. Harada. *R2-Dreamer: Redundancy-Reduced World Models without Decoders or Augmentation*. ICLR, 2026. arXiv:2603.18202. Code: <https://github.com/NM512/r2dreamer>.
- [19] NM512. *dreamerv3-torch: PyTorch Implementation of DreamerV3*. GitHub repository, <https://github.com/NM512/dreamerv3-torch>.